Laboratorio 01

Esercizio 1: Processi vs Thread

Obiettivo: Modificare **reactivity-processes.c** e **reactivity-threads.c** in modo da rendere meno reattiva la sezione multi-process, a vantaggio della sezione multi-thread, e verificare l'aumento dello speedup.

Differenza sfruttata:
- Con **fork()** il processo figlio riceve una copia dell'intera memoria del padre. Su Linux la copia avviene in **copy-on-write**: una pagina viene duplicata solo quando il figlio la scrive;
- I thread di un processo condividono la memoria, quindi non hanno questo costo.

Come l'ho sfruttata:
- Il padre alloca e azzera un grosso blocco di memoria (256 MB) prima di iniziare la misurazione, cosi le pagine sono realmente mappate;
- La funzione **do_work()**, identica nei due file, scrive un byte per ogni pagina del blocco:
  - Nei **thread** la memoria è già condivisa, quindi le scritture costano quasi nulla.
  - Nei **processi** figli ogni scrittura provoca un page fault e la copia della pagina (copy-on-write). Inoltre fork() deve copiare le tabelle delle pagine del padre.

Codice: Definizioni e do_work(), identiche in reactivity-processes.c e reactivity-thread.c:

```
#define MEM_SIZE (256UL * 1024 * 1024)  // 256 MB di memoria del
padre
#define PAGE_SIZE 4096

static char *big_mem;

void do_work() {
        unsigned long i;
        for (i = 0; i < MEM_SIZE; i += PAGE_SIZE)
                big_mem[i] = 1;
}
```

Aggiunta nel main di entrambi i file, dopo il parsing di N:

```
// il padre alloca e tocca tutta la memoria PRIMA di misurare,
// così le pagine sono realmente mappate (e verranno condivise con
fork)
big_mem = malloc(MEM_SIZE);
if (big_mem == NULL) {
        fprintf(stderr, "Can't allocate memory\n");
        exit(EXIT_FAILURE);
}
memset(big_mem, 0, MEM_SIZE);
```

Il resto dei due programmi resta invariato.

Spiegazione del codice:

Definizioni
- **MEM_SIZE** è la quantità di memoria che il padre mette da parte (256 MB), abbastanza grande da rendere visibile il costo del **fork()**.

- **PAGE_SIZE** è la dimensione di una pagina di memoria virtuale (4096 byte). Serve a saltare da una pagina all'altra.
- **big_mem** è un puntatore globale al blocco di memoria. E' globale per essere visibile sia a **do_work()** sia al **main**.

Preparazione del **main**
- **malloc(MEM_SIZE)** riserva il blocco. Se restituisce NULL, il programma termina con un errore.
- **memset(big_mem, 0, MEM_SIZE)** scrive zeri su tutto il blocco. Questo passaggio è importante: **malloc** da solo riserva solo indirizzi virtuali, mentre **memset** forza il sistema a mapparli davvero in memoria fisica.
- Tutto questo avviene prima di **begin(&t)**, quindi non rientra nel tempo misurato.

do_work()
- Il ciclo avanza di **PAGE_SIZE** byte alla volta e scrive **1** in un byte per ogni pagina, in modo da toccarle tutte con il minimo lavoro.
- Il lavoro in sé è trascurabile: quello che conta è l'**effetto sulla memoria**.

Cosa succede nei due casi:

Processi (**fork()**)
- Il figlio parte con le stesse pagine del padre, condivise in sola lettura (copy-on-write).
- Alla prima scrittura su ciascuna pagina, il sistema operativo la duplica per il figlio: 256 MB/4 KB = 65.536 pagine da copiare a ogni figlio.
- Inoltre **fork()** stesso deve copiare le tabelle delle pagine del padre, e il costo cresce con la memoria mappata.

Thread (**pthread_create**)
- Il thread usa la stessa memoria del processo, quindi nessuna copia e nessun page fault da copy-on-write.
- **do_work()** scrive direttamente sulle pagine già presenti.

Compilazione ed esecuzione:

```
gcc -o reactivity-processes-es1 reactivity-processes-es1.c performance.c -lrt -lm

gcc -o reactivity-threads-es1 reactivity-threads-es1.c performance.c -lpthread -lrt -lm

./reactivity-processes-es1 200

./reactivity-threads-es1 200
```

Risultati:

Misurato sulla macchina di prova, con 2 core.

	Processi	Thread	Speedup
Versione originale (N = 1000)	197 µs	67 µs	≈ 2,9
Versione modificata (N = 200)	258.619 µs	2.933 µs	≈ 88

Conclusione: Facendo lavorare i processi figli su una grande quantità di memoria ereditata dal padre, il costo del copy-on-write fa crescere molto il tempo dei processi. I thread, che condividono la memoria, restano veloci, e lo speedup passa da circa 3 a circa 88.

Esercizio 2: Accesso concorrente a variabili condivise

Obiettivo: Capire cosa succede quando più thread accedono in scrittura a una variabile condivisa in concorrenza, e risolvere il problema senza meccanismi di sincronizzazione, mantenendo la semantica originale.

Problema: La sorgente di partenza è **concurrent_threads.c**, compilato con:

gcc -o concurrent_threads concurrent_threads-c -lpthread

Lancia **N thread in parallelo**, e ognuno aggiunge **M** volte un valore **V** a una **variabile condivisa inizializzata a 0**. Alla fine la variabile dovrebbe valere **N*M*V**: succede sempre?
NO. Tutti i thread eseguono **shared_variable += v** sulla stessa variabile, e questa i**struzione non è atomica**: è composta da lettura, somma e scrittura. Due thread possono quindi leggere lo stesso valore e uno dei due aggiornamenti va perso (**race condition**).

Specifiche dell'esercizio:
- **N** thread effettuano in parallelo **M** incrementi di valore **V**.
- Al termine, il main thread verifica che tali incrementi equivalgono complessivamente a **N*M*V**.
- Va modificato **concurrent_thread.c** di conseguenza.
- **Suggerimento**: lavorare sulle strutture dati per evitare accessi concorrenti in scrittura.

Soluzione 1: una locazione di memoria per thread (**sol_concurrent_threads.c**)
Ogni thread incrementa una locazione di memoria diversa, e alla fine il main thread somma tutti i valori. Non dovrebbero risultare add perse.

```
unsigned long int* partial_sums;   // un contatore per ogni thread

void* thread_work(void *arg) {
        int thread_idx = *((int*)arg); // indice del thread (e della sua
locazione)
        int i;
        for (i = 0; i < m; i++)
                partial_sums[thread_idx] += v;
        return NULL;
}
```

Nel **main**:

```
partial_sums = (unsigned long int*)calloc(n, sizeof(unsigned long int));
pthread_t* threads = (pthread_t*)malloc(n * sizeof(pthread_t));
int* thread_ids = (int*)malloc(n * sizeof(int)); // un argomento distinto per
thread

for (i = 0; i < n; i++) {
        thread_ids[i] = i;
        pthread_create(&threads[i], NULL, thread_work, &thread_ids[i]);
}
for (i = 0; i < n; i++)
        pthread_join(threads[i], NULL);

// il main thread somma tutti i valori
unsigned long int shared_variable = 0;
for (i = 0; i < n; i++)
        shared_variable += partial_sums[i];
```

Spiegazione:
- Ogni thread riceve un indice diverso, letto da **thread_ids[i]**, e scrive solo in **partial_sums[thread_idx]**. Nessuna locazione è scritta da due thread, quindi non c'è race condition e non servono lock.
- Il main aspetta tutti i thread con **pthread_join** e solo allora somma i contatori, quindi legge valori ormai definitivi.

Compilazione ed esecuzione:

```
gcc -o sol_concurrent_threads sol_concurrent_threads.c -lpthread

./sol_concurrent_threads <N> <M> <V>
```

Cosa succede con &i al posto di &thread_ids[i]:
Il risultato è sbagliato e cambia a ogni esecuzione:
- Non si può avere alcuna garanzia su quando verra eseguita l'istruzione **int thread_idx = *((int*)arte);**.
- Nel mentre, il valore nella locazione di memoria puntata da **arg**, cioè il contatore **i** del ciclo nel **main**, può cambiare.
- Più thread possono quindi avere la stessa identità (**thread_idx**) e scrivere sulla stessa locazione.

Soluzione alternativa: variabili locali (sol2_concurrent_threads.c)
Ogni thread lavora su variabili locali e restituisce il valore tramite **pthread_exit**. Il main raccoglie i valori tramite **pthread_join** e li somma.

```
void* thread_work(void *arg) {
        unsigned long int local_sum = 0; // variabile locale del thread
        int i;
        for (i = 0; i < m; i++)
                local_sum += v;
        pthread_exit((void*)(uintptr_t)local_sum); // restituisce il valore al join
}
```

Nel **main**:

```
unsigned long int shared_variable = 0;
for (i = 0; i < n; i++) {
        void* ret;
        pthread_join(threads[i], &ret);
        shared_variable += (unsigned long int)(uintptr_t)ret;
}
```

Spiegazione:
- Ogni thread accumula su una variabile **locale** (sul suo stack, privata) e la restituisce con **pthread_exit**.
- Il main la recupera tramite il secondo argomento di **pthread_join** e la somma. Non serve né l'array dei contatori né un argomento da passare ai thread.

Compilazione ed esecuzione:

```
gcc -o sol2_concurrent_threads sol2_concurrent_threads.c
-lpthread

./sol2_concurrent_threads <N> <M> <V>
```

Risultati:

Programma	Risultato (N=1000, M=10000, V=1)
Originale	9.298.690 – 9.686.931 (add perse)
Soluzione 1 (`sol_concurrent_threads`)	10.000.000 (tre esecuzioni su tre)
Soluzione alternativa (`sol2_concurrent_threads`)	10.000.000 (tre esecuzioni su tre)
Variante con `&i`	9.348.728 – 9.540.841 (add perse)

Conclusione: Per evitare la **race condition senza meccanismi di sincronizzazione** basta che nessuna locazione sia scritta da più thread: ogni thread lavora su dati propri e il main combina i risultato dopo la **pthread_join**.

INDICE

Laboratorio 01 **1**

Esercizio 1: Processi vs Thread 1

Esercizio 2: Accesso concorrente a variabili condivise 3